

```

    x[i]=y[i];
    x[i+4]=y1[i]*d1;
    x[i+8]=y2[i]*d2;
    x[i+12]=y12[i]*d1d2;
}
for (i=0;i<16;i++) {           Matrix-multiply by the stored table.
    xx=0.0;
    for (k=0;k<16;k++) xx += wt[i][k]*x[k];
    cl[i]=xx;
}
l=0;
for (i=0;i<4;i++)             Unpack the result into the output table.
    for (j=0;j<4;j++) c[i][j]=cl[l++];
}

```

The implementation of equation (3.6.6), which performs a bicubic interpolation, gives back the interpolated function value and the two gradient values, and uses the above routine `bucuf`, is simply:

```

interp_2d.h void bcuint(VecDoub_I &y, VecDoub_I &y1, VecDoub_I &y2, VecDoub_I &y12,
    const Doub x1l, const Doub x1u, const Doub x2l, const Doub x2u,
    const Doub x1, const Doub x2, Doub &ansy, Doub &ansy1, Doub &ansy2) {
Bicubic interpolation within a grid square. Input quantities are y,y1,y2,y12 (as described in
bucuf); x1l and x1u, the lower and upper coordinates of the grid square in the 1 direction;
x2l and x2u likewise for the 2 direction; and x1,x2, the coordinates of the desired point for
the interpolation. The interpolated function value is returned as ansy, and the interpolated
gradient values as ansy1 and ansy2. This routine calls bucuf.
    Int i;
    Doub t,u,d1=x1u-x1l,d2=x2u-x2l;
    MatDoub c(4,4);
    bucuf(y,y1,y2,y12,d1,d2,c);           Get the c's.
    if (x1u == x1l || x2u == x2l)
        throw("Bad input in routine bcuint");
    t=(x1-x1l)/d1;                         Equation (3.6.4).
    u=(x2-x2l)/d2;
    ansy=ansy2=ansy1=0.0;
    for (i=3;i>=0;i--) {                  Equation (3.6.6).
        ansy=t*ansy+((c[i][3]*u+c[i][2])*u+c[i][1])*u+c[i][0];
        ansy2=t*ansy2+(3.0*c[i][3]*u+2.0*c[i][2])*u+c[i][1];
        ansy1=u*ansy1+(3.0*c[i][3]*t+2.0*c[i][2])*t+c[i][1];
    }
    ansy1 /= d1;
    ansy2 /= d2;
}

```

You can combine the best features of bicubic interpolation and bicubic splines by using splines to compute values for the necessary derivatives at the grid points, storing these values, and then using bicubic interpolation, with an efficient table-searching method, for the actual function interpolations. Unfortunately this is beyond our scope here.

CITED REFERENCES AND FURTHER READING:

- Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at <http://www.nrl.com/aands>, §25.2.
- Kinahan, B.F., and Harm, R. 1975, "Chemical Composition and the Hertzprung Gap," *Astrophysical Journal*, vol. 200, pp. 330–335.

- Johnson, L.W., and Riess, R.D. 1982, *Numerical Analysis*, 2nd ed. (Reading, MA: Addison-Wesley), §5.2.7.
- Dahlquist, G., and Björck, A. 1974, *Numerical Methods* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2003 (New York: Dover), §7.7.

3.7 Interpolation on Scattered Data in Multidimensions

We now leave behind, if with some trepidation, the orderly world of regular grids. Courage is required. We are given an arbitrarily scattered set of N data points $(\mathbf{x}_i, y_i)$, $i = 0, \dots, N-1$ in n -dimensional space. Here $\mathbf{x}_i$ denotes an n -dimensional vector of independent variables, $(x_{1i}, x_{2i}, \dots, x_{ni})$, and y_i is the value of the function at that point.

In this section we discuss two of the most widely used *general* methods for this problem, *radial basis function (RBF)* interpolation, and *kriging*. Both of these methods are expensive. By that we mean that they require $O(N^3)$ operations to initially digest a set of data points, followed by $O(N)$ operations for each interpolated value. Kriging is also able to supply an error estimate — but at the rather high cost of $O(N^2)$ per value. Shepard interpolation, discussed below, is a variant of RBF that at least avoids the $O(N^3)$ initial work; otherwise these workloads effectively limit the usefulness of these general methods to values of $N \lesssim 10^4$. It is therefore worthwhile for you to consider whether you have any other options. Two of these are

- If n is not too large (meaning, usually, $n = 2$), and if the data points are fairly dense, then consider triangulation, discussed in §21.6. Triangulation is an example of a finite element method. Such methods construct some semblance of geometric regularity and then exploit that construction to advantage. Mesh generation is a closely related subject.
- If your accuracy goals will tolerate it, consider moving each data point to the nearest point on a regular Cartesian grid and then using Laplace interpolation (§3.8) to fill in the rest of the grid points. After that, you can interpolate on the grid by the methods of §3.6. You will need to compromise between making the grid very fine (to minimize the error introduced when you move the points) and the compute time workload of the Laplace method.

If neither of these options seem attractive, and you can't think of another one that is, then try one or both of the two methods that we now discuss. RBF interpolation is probably the more widely used of the two, but kriging is our personal favorite. Which works better will depend on the details of your problem.

The related, but easier, problem of *curve interpolation* in multidimensions is discussed at the end of this section.

3.7.1 Radial Basis Function Interpolation

The idea behind RBF interpolation is very simple: Imagine that every known point j “influences” its surroundings the same way in all directions, according to

some assumed functional form $\phi(r)$ — the radial basis function — that is a function only of radial distance $r = |\mathbf{x} - \mathbf{x}_j|$ from the point. Let us try to approximate the interpolating function everywhere by a linear combination of the ϕ 's, centered on all the known points,

$$y(\mathbf{x}) = \sum_{i=0}^{N-1} w_i \phi(|\mathbf{x} - \mathbf{x}_i|) \quad (3.7.1)$$

where the w_i 's are some unknown set of weights. How do we find these weights? Well, we haven't used the function values y_i yet. The weights are determined by requiring that the interpolation be exact at all the known data points. That is equivalent to solving a set of N linear equations in N unknowns for the w_i 's:

$$y_j = \sum_{i=0}^{N-1} w_i \phi(|\mathbf{x}_j - \mathbf{x}_i|) \quad (3.7.2)$$

For many functional forms ϕ , it can be proved, under various general assumptions, that this set of equations is nondegenerate and can be readily solved by, e.g., LU decomposition (§2.3). References [1,2] provide entry to the literature.

A variant on RBF interpolation is *normalized radial basis function (NRBF)* interpolation, in which we require the sum of the basis functions to be unity or, equivalently, replace equations (3.7.1) and (3.7.2) by

$$y(\mathbf{x}) = \frac{\sum_{i=0}^{N-1} w_i \phi(|\mathbf{x} - \mathbf{x}_i|)}{\sum_{i=0}^{N-1} \phi(|\mathbf{x} - \mathbf{x}_i|)} \quad (3.7.3)$$

and

$$y_j \sum_{i=0}^{N-1} \phi(|\mathbf{x}_j - \mathbf{x}_i|) = \sum_{i=0}^{N-1} w_i \phi(|\mathbf{x}_j - \mathbf{x}_i|) \quad (3.7.4)$$

Equations (3.7.3) and (3.7.4) arise more naturally from a Bayesian statistical perspective [3]. However, there is no evidence that either the NRBF method is consistently superior to the RBF method, or vice versa. It is easy to implement both methods in the same code, leaving the choice to the user.

As we already mentioned, for N data points the one-time work to solve for the weights by LU decomposition is $O(N^3)$. After that, the cost is $O(N)$ for each interpolation. Thus $N \sim 10^3$ is a rough dividing line (at 2007 desktop speeds) between “easy” and “difficult.” If your N is larger, however, don't despair: There are *fast multipole methods*, beyond our scope here, with much more favorable scaling [1,4,5]. Another, much lower-tech, option is to use *Shepard interpolation* discussed later in this section.

Here are a couple of objects that implement everything discussed thus far. `RBF_fn` is a virtual base class whose derived classes will embody different functional forms for $\text{rbf}(r) \equiv \phi(r)$. `RBF_interp`, via its constructor, digests your data and solves the equations for the weights. The data points $\mathbf{x}_i$ are input as an $N \times n$ matrix, and the code works for any dimension n . A boolean argument `nrbf` inputs whether NRBF is to be used instead of RBF. You call `interp` to get an interpolated function value at a new point $\mathbf{x}$.

```
struct RBF_fn {
```

Abstract base class template for any particular radial basis function. See specific examples below.

```
    virtual Doub rbf(Doub r) = 0;
};
```

```
struct RBF_interp {
```

Object for radial basis function interpolation using n points in dim dimensions. Call constructor once, then `interp` as many times as desired.

```
    Int dim, n;
    const MatDoub &pts;
    const VecDoub &vals;
    VecDoub w;
    RBF_fn &fn;
    Bool norm;

    RBF_interp(MatDoub_I &ptss, VecDoub_I &valss, RBF_fn &func, Bool nrbf=false)
    : dim(ptss.ncols()), n(ptss.nrows()) , pts(ptss), vals(valss),
    w(n), fn(func), norm(nrbf) {
```

Constructor. The $n \times \text{dim}$ matrix `ptss` inputs the data points, the vector `valss` the function values. `func` contains the chosen radial basis function, derived from the class `RBF_fn`. The default value of `nrbf` gives RBF interpolation; set it to 1 for NRBF.

```
    Int i,j;
    Doub sum;
    MatDoub rbf(n,n);
    VecDoub rhs(n);
    for (i=0;i<n;i++) {                                Fill the matrix  $\phi(|\mathbf{r}_i - \mathbf{r}_j|)$  and the r.h.s. vector.
        sum = 0.;
        for (j=0;j<n;j++) {
            sum += (rbf[i][j] = fn.rbf(rad(&pts[i][0],&pts[j][0])));
        }
        if (norm) rhs[i] = sum*vals[i];
        else rhs[i] = vals[i];
    }
    LUdcmp lu(rbf);                                    Solve the set of linear equations.
    lu.solve(rhs,w);
}
```

```
Doub interp(VecDoub_I &pt) {
```

Return the interpolated function value at a dim -dimensional point `pt`.

```
    Doub fval, sum=0., sumw=0.;
    if (pt.size() != dim) throw("RBF_interp bad pt size");
    for (Int i=0;i<n;i++) {                            Sum over all tabulated points.
        fval = fn.rbf(rad(&pt[0],&pts[i][0]));
        sumw += w[i]*fval;
        sum += fval;
    }
    return norm ? sumw/sum : sumw;
}
```

```
Doub rad(const Doub *p1, const Doub *p2) {
```

Euclidean distance.

```
    Doub sum = 0.;
    for (Int i=0;i<dim;i++) sum += SQR(p1[i]-p2[i]);
    return sqrt(sum);
}
};
```

3.7.2 Radial Basis Functions in General Use

The most often used radial basis function is the *multiquadric* first used by Hardy, circa 1970. The functional form is